

Learning Geometric Structures from Data

Lecture 2 | AI meets geometry

XXII Escola de Geometria Diferencial – Teresina, Piauí

Tomás S. R. Silva

IMECC — Universidade Estadual de Campinas

This lecture — and how it runs

Lecture 1: three ML paradigms.

Today: supervised learning — representing and fitting geometric structures.

Three topics, interleaved with hands-on notebooks:

- | | |
|--|-----------------------------------|
| 1. Euler's formula for polytopes | 15 min + 10 min practical session |
| 2. Ricci-flat Calabi–Yau metrics | 15 min + 10 min practical session |
| 3. G_2 -structures on 7-manifolds (and beyond) | 10 min discussion |



The image shows the homepage of the XXII Escola de Geometria Diferencial (EGD) website. The header features the site's logo and name, a navigation menu with links to HOME, ABOUT, SCIENTIFIC PROGRAM, BOOKS, and REGISTRATION, and a search icon. The main content area includes a large graphic of stacked coins, the event title "XXII Escola de Geometria Diferencial", the location and dates "Teresina, July 13-17, 2026", social media icons for Instagram and Facebook, and a "Register Now" button. A red arrow points from the bottom right towards the "BOOKS" link in the navigation menu. The footer contains a repeating pattern of the event title.

XXII EGD

<https://egd.ufpi.edu.br/>

HOME ABOUT ▾ SCIENTIFIC PROGRAM ▾ BOOKS REGISTRATION 🔍



XXII Escola de Geometria Diferencial
Teresina, July 13-17, 2026

Register Now →

Geometria Diferencial | **XXII Escola de Geometria Diferencial** | XXII Escola



Edward Hirst, Henrique N. Sá Earp & Tomás S. R. Silva

Brief Description

"AI meets Geometry" is a hands-on minicourse introducing modern artificial intelligence as a practical tool for exploration, computation, and discovery in differential geometry. Participants will learn how neural networks, physics-informed methods, optimization, and reinforcement learning can be used to approximate geometric structures, study metrics and differential forms, investigate geometric flows, and search for new examples and conjectures, while keeping clear the essential distinction between numerical discovery and mathematical verification. No prior machine-learning background is assumed: the course builds conceptual foundations alongside concrete geometric case studies, including special metrics, neural PDE methods, and AI-assisted discovery. Code will be provided throughout, giving attendees working notebooks that they can modify, extend, and adapt directly to questions arising in their own research.

Download



README

MIT license

Lecture	Notebook	Open in Colab
Lecture 2	Part I	Open in Colab
Lecture 2	Part II	Open in Colab
Lecture 3	Full notebook	Open in Colab
Lecture 4	Part I	Open in Colab

AI meets Geometry

XXII ESCOLA DE GEOMETRIA DIFERENCIAL
FOUR-LECTURE MINICOURSE

Learn how AI can approximate geometric structures, solve variational PDEs, and search for new mathematical examples.



1 AI Overview

Map supervised, unsupervised, and reinforcement learning onto geometric approximation, pattern-finding, and search.



2 Learning Geometric Structures

Train neural networks to learn metrics and differential forms while enforcing symmetry, positivity, and geometric identities.



3 Neural PDEs & Geometric Flows

Encode geometric PDEs in neural losses and flow an immersed torus toward the Willmore minimiser.



4 Discovery & Verification

Search line arrangements and other geometric spaces with reinforcement learning, then verify discoveries exactly.

Packages

No packages published
[Publish your first package](#)

Contributors 2

edhirst Ed Hirst

TomasSilva Tomás Silva

Languages

Jupyter Notebook 100.0%

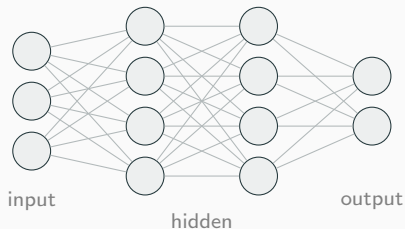
I. Euler's formula

The machine, in one slide

A neural network is a function $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

- Layers: $v_k = \sigma(W_k v_{k-1} + b_k)$,
parameters $\theta = \{W_k, b_k\}$.
- σ a nonlinearity (ReLU, GELU).
- $f_\theta(\mathbf{x}) =$
 $b_N + W_N \sigma(b_{N-1} + W_{N-1} \sigma(\cdots b_1 + W_1 \mathbf{x}))$
- Train by descending a loss: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$,
with $\nabla_\theta \mathcal{L}$ from **autodiff**.
- Universal approximation: it can fit **any** nice
enough map.

We reuse this same machine for all three topics.



Two ways to build the loss

This distinction organises the whole lecture.

(A) Supervised — learn from labelled examples

Given pairs (x, y) , minimise $\|f_\theta(x) - y\|^2$.

Both of today's labs: Euler's formula, and the Calabi–Yau metric (Topic II).

(B) Self-supervised — no labels, only equations

Assemble the loss from the equations that f_θ must satisfy.

Same machinery; different source of the training signal.

Why not MNIST?

- ML's "hello world": classify handwritten digits (MNIST) — human, noisy, finite.
- Geometry gives us **better** toy data: **exact**, **cheap**, **unlimited**, **noiseless**.
- So our first network learns a **theorem** instead of a digit.

Our "MNIST" = **polytopes and their invariants**.

The theorem we will (re)discover

Euler's formula for a convex 3-polytope:

$$V - E + F = 2.$$

Euler–Poincaré for a d -polytope, with face numbers $f_0, \dots, f_{d-1}$:

$$\sum_{i=0}^{d-1} (-1)^i f_i = 1 - (-1)^d.$$

χ is a **topological invariant** — the same for every polytope of that dimension.

The task

1. Sample many random polytopes; record $(f_0, f_1, f_2) = (V, E, F)$.
2. Do not tell the network the formula.
3. Train, by MSE, the map $f_\theta : (V, E) \mapsto F$.
4. Ask: what did it learn?

A deliberately trivial target — because we already know the answer.

What it learns — and why we care

- It recovers $F = E - V + 2$: weights $\approx (-1, +1)$, bias ≈ 2 . The network discovered Euler's formula.
- Noiseless data $\Rightarrow$ we can check **exactly** that it got the maths.
- **Template established:** encode as vectors $\rightarrow$ pick a loss $\rightarrow$ train $\rightarrow$ **verify**.

The real version

Reflexive polytopes (Kreuzer–Skarke) $\leftrightarrow$ Calabi–Yau 3-folds; ML predicts Hodge numbers where **no closed formula is known**. The toy problem is the gateway to an active programme — and to Topic II.

E.g.: Machine learning Sasakian and G_2 topology on contact Calabi-Yau 7-manifolds – D. Aggarwal, Y-H He, E. Heyes, E. Hirst, and H. Sá Earp, _____

Scaling up: does it find Euler–Poincaré?

- Same trick in dimension d : face vector $(f_0, \dots, f_{d-1})$, predict one entry from the rest.
- For 4-polytopes, a single linear neuron recovers

$$f_3 = f_0 - f_1 + f_2,$$

i.e. the coefficients $(+1, -1, +1)$ of the Euler–Poincaré relation.

- The lesson generalises: with the right encoding a linear model reads off **exact** identities — and nonlinear models can **conjecture** new ones where none is known.

Practical I

Open `lecture_2-part-I.ipynb` (~10 min)

Sample random polytopes; compute (V, E, F) .

Train a small **PyTorch** net $(V, E) \mapsto F$.

Read off the weights — did it find Euler?

II. Ricci-flat CY metrics

The Calabi–Yau metric problem

- A CY 3-fold is Kähler with $c_1 = 0 \Rightarrow$ a **unique** Ricci-flat metric (Calabi–Yau theorem).
- **But:** no closed-form Ricci-flat metric is known on **any** compact CY 3-fold.
- Wanted for Physics (Yukawa couplings, moduli) **and** Mathematics (SYZ, mirror symmetry).

So we **approximate** numerically — and then **learn** it.

What we are trying to learn

The metric is Kähler, so it comes from a single real potential K :

$$g_{a\bar{b}} = \partial_a \partial_{\bar{b}} K, \quad \text{Ricci-flat} \iff \underbrace{\omega^3 = c \Omega \wedge \bar{\Omega}}_{\text{Monge-Ampère}}.$$

- Target: the metric $g_{a\bar{b}}$ (a 3×3 Hermitian matrix at each point), or its **determinant** $g = \det g_{a\bar{b}}$ — a single real function.
- **Benchmark:** the Fermat quintic $\{\sum_i z_i^5 = 0\} \subset \mathbb{P}^4$.

Where the labels come from: Donaldson's algorithm

A classical scheme produces **balanced metrics** $g^{(k)}$:

- algebraic Kähler potential $K = \frac{1}{k\pi} \ln \sum h^{\alpha\bar{\beta}} s_{\alpha} \bar{s}_{\beta}$, with $\{s_{\alpha}\}$ degree- k polynomials and h fixed by iteration;
- as $k \rightarrow \infty$, $g^{(k)} \rightarrow$ the Ricci-flat metric.

S. Donaldson, *Some numerical results in complex differential geometry*, arXiv:0512625

This is our **label generator** — but its cost grows **factorially** in k :

$$k = 12 \approx 50 \text{ hours}, \quad k = 20 \approx 35 \text{ years.}$$

$\Rightarrow$ let a network do the heavy lifting.

The supervised task — the Euler recipe again

Exactly construction **(A)**, identical in shape to the polytope lab:

- **Encode** each point $p = (z_0, \dots, z_4)$ by its real & imaginary parts.
- **Target** = the determinant $g^{(k)}|_p$ from Donaldson (a real number).
- **Train** a network by MSE on labelled pairs

$$\{p, g^{(l)}|_p\} \longmapsto g^{(k)}|_p, \quad k > l.$$

- Learn from a **small** sample; predict the metric **everywhere**.

How we verify: the σ -measure

How good is the learned metric? Measure its **Ricci-flatness** directly:

$$\sigma = \frac{1}{\text{Vol}} \int_X \left| 1 - \frac{\omega^3 / \text{Vol}_K}{\Omega \wedge \bar{\Omega} / \text{Vol}_{CY}} \right| \xrightarrow{\text{Ricci-flat}} 0.$$

- Encodes the Monge–Ampère equation; computable from the **determinant** alone.
- Our “did it recover Euler?” check — here, “did it drive σ down?”.

Does it work?

- Predicted vs. true determinant cluster on the line $y = x$.
- Trained on a **small** sample, the network's $\hat{\sigma}$ lands within a few % of Donaldson's σ — and it extends to all nine metric components.
- A decaying-exponential fit in k , $g^{(k)}|_p \approx a_p - b_p e^{-c_p k}$, removes the need for costly high- k labels.
- Net effect: $\sim 50\text{--}75\times$ faster than Donaldson's algorithm alone.

Ashmore, He, Ovrut — **Machine Learning Calabi–Yau Metrics**, arXiv:1910.08605.

Beyond labels: a loss from geometry

Supervised learning needs **labels** — and Donaldson data is expensive to make.

A more powerful idea (construction **B**): do not imitate any reference metric — build the loss **directly from the equations** the metric must satisfy, and minimise that.

No ground truth required.

The state of the art: cymetric's custom loss

Learn a **correction** to Fubini–Study, $g = g_{\text{FS}} + i\partial\bar{\partial}\varphi$, minimising a five-term **geometric** loss:

$$\mathcal{L} = \mathcal{L}_{\text{MA}} + \mathcal{L}_{\text{dJ}} + \mathcal{L}_{\text{trans}} + \mathcal{L}_{\text{Ricci}} + \mathcal{L}_{\text{K-class}}$$

(Monge–Ampère, Kähler, patching, Ricci, Kähler class — Larfors–Lukas–Ruehle–Schneider.)

- Label-free: the geometry **is** the supervision.
- Exactly the tool the G_2 construction builds on — **next**.

Practical II

Open `lecture_2-part-II.ipynb` (~10 min)

Build a labelled dataset on the quintic: points $\rightarrow$ metric.

Train a **supervised** network to predict it, by MSE.

Verify with predicted-vs-true and the σ -measure.

III. G_2 -structures

- $SU(3)$ holonomy (Calabi–Yau) is comparatively well understood.
- G_2 -holonomy 7-manifolds: central in M-theory, far less understood, **no explicit metrics**.
- They lack an analogue of Dolbeault cohomology — moduli are much harder.

Plan: run the **same playbook**, one structure up.

A G_2 -structure is a non-degenerate 3-form φ , modelled on

$$\varphi_0 = e^{123} + e^{145} + e^{167} + e^{246} - e^{257} - e^{347} - e^{356} \in \Lambda^3(\mathbb{R}^7)^*.$$

- φ induces a metric g_φ , an orientation and a Hodge star $*_\varphi$.
- In fact g_φ is recovered **algebraically** from φ : $(g_\varphi)_{ij} = B_{ij}/(\det B)^{1/9}$.
- **Torsion-free** $\iff d\varphi = 0$ and $d*_\varphi\varphi = 0 \iff \text{Hol}(g_\varphi) \subseteq G_2$ (Ricci-flat).

A computable bridge: contact Calabi–Yau

A cCY 7-manifold is an S^1 -bundle over a CY 3-fold; real-algebraic, hence computable. It carries a natural G_2 -structure **built from the base**:

$$\varphi = \eta \wedge \omega + \operatorname{Re} \Upsilon, \quad \psi = *\varphi = \frac{1}{2} \omega \wedge \omega + \eta \wedge \operatorname{Im} \Upsilon.$$

Coclosed but **not** closed:

$$d\psi = 0, \quad d\varphi = \omega \wedge \omega \neq 0.$$

A concrete starting structure — that **reuses the CY metric we just learned**.

$$\begin{array}{ccc} K_f & \longrightarrow & S^9 \\ S^1 \downarrow & & \downarrow \\ V & \longrightarrow & \mathbb{P}^4(w) \end{array}$$

The pipeline

1. **Stage 1.** cymetric $\Rightarrow$ Ricci-flat metric on the quintic base.
2. **Stage 2.** Build $(\varphi, \psi, g_\varphi)$ **pointwise** on the link ($\sim 900\text{k}$ samples).
3. **Stage 3.** Train a network on the 35 components of φ (and 28 of g_φ).

Fully **supervised** — construction **(A)**: test $\text{MSE} \approx 1.5 \times 10^{-6}$.

Verification: trust, but check

- Algebraic identity across the dataset:

$$\varphi \wedge \psi = 7 \operatorname{vol}(g_\varphi) \quad (\text{ratio} \approx 7).$$

- Torsion via a numerical exterior derivative on the learned φ :

$$d\varphi \approx \omega \wedge \omega, \quad d\psi \approx 0$$

(up to a controlled discretisation plateau).

First neural approximation of a G_2 -structure on a Calabi–Yau link.

- The learned metric is **initial data** for a geometric flow (Laplacian coflow) towards a **torsion-free** G_2 -structure.
- Evolving PDEs with networks (PINNs) = **Lecture 3 (EH)**.
- The dream: genuinely **new** examples of G_2 -holonomy metrics.

Topic III is theory — where to read more

The G_2 construction and every result quoted above are from our paper:

- E. Heyes, E. Hirst, H. N. Sá Earp, _____,
Neural and numerical methods for G_2 -structures on contact Calabi–Yau
7-manifolds, arXiv:2602.12438 (2026).
- Code: `github.com/TomasSilva/LearningG2`.
- CY-metric tooling: `cymetric` — Larfors, Lukas, Ruehle, Schneider,
arXiv:2111.01436.

No notebook for this part — take it as the horizon we are heading toward.

One recipe, three problems

1. **Encode** the structure as tensors / vectors.
2. **Choose a loss:** (A) supervised — Euler, the CY metric, G_2 tensors; (B) self-supervised — the frontier method (cymetric).
3. **Train** with autodiff on the chosen signal.
4. **Verify** with an **independent** check (Euler's coefficients, the σ -measure, $\varphi \wedge \psi = 7 \text{ vol}$).

Next door — **Lecture 3:** evolving these structures with neural PDEs.

Thank You!

Find me @:

`tomas@ime.unicamp.br`

`www.ime.unicamp.br/~tomas`

`mathstodon.xyz/@tomassilva`